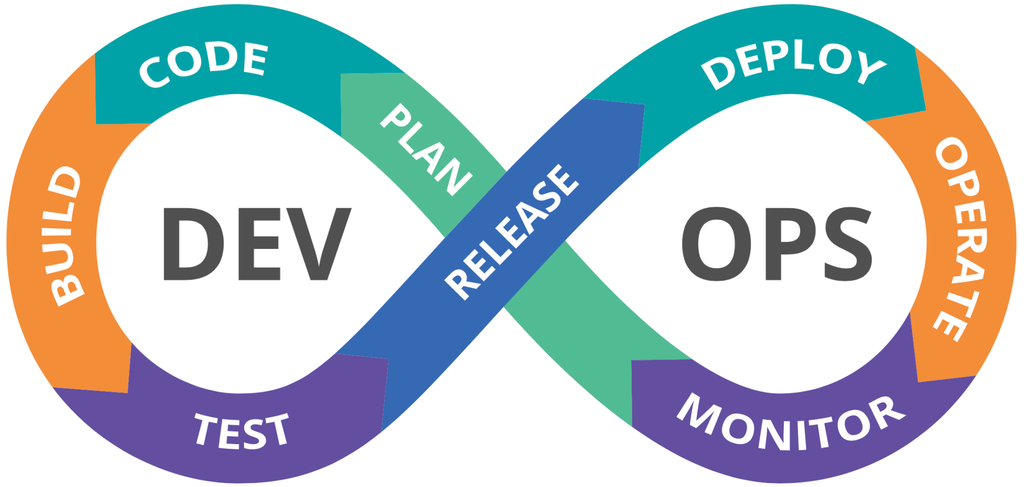
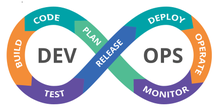


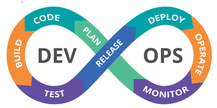
Tehničke prakse za uvođenje principa toka
Omogućavanje i praktikovanje kontinualne integracije

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



Tehničke prakse za uvođenje principe toka

- Kontinualna isporuka (*Continuous Delivery*) podrazumeva
  - uspostavljanje *deployment pipeline-a*
  - omogućavanje brzog i pouzdanog automatizovanog testiranja
  - **omogućavanje i praktikovanje kontinualne integracije**
  - automatizaciju i upravljanje razvojem koji omogućava isporuku uz male rizike (*low risk deployment*)

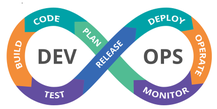


Zašto je potrebna kontinualna integracija?

- Prethodno smo opisali važnost automatizovanih testova i kako brza povratna informacija o tome da li testovi prolaze ili ne pomaže programerima. Automatizovano testiranje postaje tim bitnije kako se povećava broj programera i samim tim grana u kojima se razvija kod.

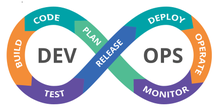
- Sama mogućnost da se kod razvija u nezavisnim granama omogućava paralelni rad na razdvojenim *featurima.*

- Međutim, predug rad u odvojenim granama dovodi do pojave naknadnih problema sa spajanjem koda na glavnu granu - **integracioni problemi.**



Integracioni problemi

- konflikti se moraju ručno rešavati
- dovode do potrebe da se deo koda ponovo preradi kako bi se konflikti uklonili i kod ponovo doveo u stanje da je isporučiv (da može uspešno da se instalira i pokrene)
- ovo obično zahteva da više programera koordinisano prerade međusobno konfliktne delove koda
- u tradicionalnim metodama razvoja, integracija se obavljala na samom kraju projekta (ili neke njegove veće faze) što je rezultovalo dugim vremenom potrebnim da se integracija obavi uspešno

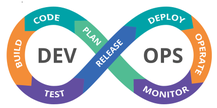


Kontinualna integracija?

- prethodno spomenuti problemi često izazivaju “negativnu spiralu” - integracija je mučna i teška, odložićemo je i raditi kasnije

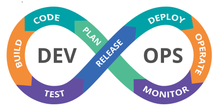
- **kontinualna integracija -** promena filozofije - neka spajanje koda na glavnu granu (naravno uz sve prethodne korake koji garantuju da taj kod ne “ruši” pipeline) treba da postane svakodnevna aktivnost svakog programera

- kontinualna integracija rešava mnogo problema



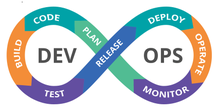
Primer problema - HP Firmware

- primer HP Firmware divizije koja je sa 400 zaposlenih uspevala da izbaci samo do dva update/a firmware-a godišnje. Ispostavilo se da samo 5% vremena se koristi na kreativni razvoj novih funkcionalnosti, a ostatak na razne neproduktivne zadatke uključujući:
  - 20% vremena su koristili na detaljno planiranje (polazeći od pretpostavke da je njihova slaba produktivnost primarno uzrokovana pogrešnim estimacijama)
  - 25% vremena je odlazilo na portovanje koda za nove proizvode (i u nove razvojne grane)
  - 10% vremena se trošilo na integrisanje koda između različitih razvojnih grana
  - 15% vremena je odlazilo na manualne testove



Predlog rešenja problema - HP Firmware

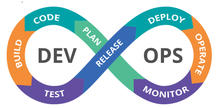
- postavljen je cilj da se vreme potrošeno na kreativni razvoj novih funkcionalnosti desetostruko uveća oslanjanjem na:
  - kontinualnu integraciju
  - automatizaciju testova
  - kreiranje simulatora hardvera kao virtuelne platforme na kojima bi se testovi mogli pokrenuti
  - omogućavanju reprodukcije greški na programerskim radnim stanicama
  - razvoju nove arhitektura koja bi omogućila da se za sve tipovi printera koristi zajednička baza koda
- Zahvaljujući kontinualnoj integraciji nakon 4 godine firmware za celu linija printera HP-a je razvijan na jednoj glavnoj grani.



Predlog rešenja problema - HP Firmware

- Ali bez automatizovanih testova kontinualna integracija ne bi bila moguća
  - uloženo je dosta truda i novca kako bi se omogućilo intenzivno testiranje
  - unit testovi su se “vrteli” na radnim mašinama programera, tri nivoa automatizovanih testova bi se obavljala nakon svakog komitovanja i dodatno na svaka 2 ili 4 sata
  - puni regresioni testovi 1 dnevno

- Rezultat konstantog integrisanja novog koda (uz sve testove):
  - broj bildova inicijalano povećan na jedan dnevno, nakon toga na do 10 dnevno
  - sa 20 komitovanja dnevno (koje je obavljala jedna “glavna” osoba), došlo se na preko 100 komitovanja dnevno - koje rade sami programeri
  - zbirno se menja/dodaje 75-100k linija koda
  - vreme regresionih testova smanjeno sa 6 nedelja na 1 dan
- Efekat na produktivnost:
  - 40% vremena se nakon toga koristilo na razvoj novih feature-a, smanjeni troškovi razvoja i do 40%

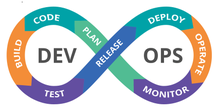


Šta je sve neophodno za kontinualnu integraciju?

- Razvoj koda u malim “paketima” i izbegavanje razvoja u dugo izdvojenim granama sa retkim spajanjima

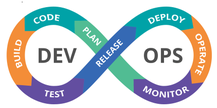
  - Razvoj velikih *feature-a* je kontraproduktivan jer dugo traje, dugo se zadržava kod u izolovanim granama, i na kraju se javlja veliki broj integracionih problema. Mali taskovi se obično realizuju brzo, sa malim izmenama koda koje se brzo spajaju (sa vrlo malo problema).

  - Strategije grananja se mogu svrstati u dve glavne grupe: optimizovanje produktivnosti pojedinca (zasniva se na privatnim granama), i optimizovanje produktivnosti tima (zasniva se na radu na zajedničkoj grani)



Šta je sve neophodno za kontinualnu integraciju?

- Usvajanje koncepta razvoja na glavnoj grani
  - Kontramera za sprečavanje predugog rada na preglomaznim izmenama koda.
  - Od programera se traži da bar jednom dnevnom svoj kod **spoje** na glavnu granu. Ovo nas tera da razvoj razlomimo u manje segmente, tako da imamo šta da komitujemo. Ukoliko spajanje rade češće - to bolje.
  - Ovo omogućava da se često “vrte” testovi koji brzo otkrivaju kod koji unosi grešku u *pipeline* i da se on odmah koriguje.
    - *gated commit* - konfigurisanje *pipeline-a* tako da se odbija sav kod koji pipeline izvodi iz *deployable* stanja. U ovom slučaju prvo se proverava da li je kod moguće spojiti, da li se bild izvršava normalno i prolaze svi automatizovani testovi i teko potom se spajanje zaista obavlja na samu glavnu granu



Kontinualna integracija ponovno menja shvatanje “urađenog”

- Na kraju razvojne faze moramo imati kod koji integrisan, testiran, radi na sistemu koji je sličan produkcionom i mora se kreirati na zahtev bildovanjem sa glavne grane